Artificial Intelligence(Lab)

Lab Task 7

Name: Abubakar Awan

SAP-ID: 54257

Section: BSCS 5-1

Summary of the Python Tic-Tac-Toe Code (Tkinter Version)

This Python program implements a **Tic-Tac-Toe game with an evaluation function and a graphical user interface (GUI)** using the **Tkinter library**. The program follows the Artificial Intelligence lab requirement of evaluating the board state using a heuristic function.

---

1. Board Representation

The game board is represented using a **3 × 3 two-dimensional list (matrix)**:

- `'x'` represents **Player X (Maximizer)**
- `'o'` represents **Player O (Minimizer)**
- `'_'` represents an **empty space**

This structure makes it easy to analyze rows, columns, and diagonals.

---

2. Evaluation Function

The program includes an **evaluation (heuristic) function** that checks the board and assigns a score based on the game result.

The function returns:

- **+10** if **Player X wins**
- **−10** if **Player O wins**
- **0** if there is **no winner or the game is a draw**

The function checks:

- All **rows**
- All **columns**
- Both **diagonals**

This evaluation helps determine the state of the game after every move.

---

3. Move Checking

Another function checks whether **any empty spaces remain on the board**.
If there are no empty spaces and no winner has been found, the game is declared a **draw**.

---

4. Graphical User Interface (GUI)

The program uses **Tkinter**, which is Python's standard GUI library.

The interface includes:

- A **3 × 3 grid of buttons** representing the Tic-Tac-Toe board.
- Each button corresponds to one cell.
- When a player clicks a button, the symbol **X or O** appears in that cell.

---

5. Game Logic

When a player clicks a cell:

1. The selected cell is updated with the current player's symbol.
2. The board is evaluated using the **evaluation function**.
3. The program checks for:
    - **Player X win**
    - **Player O win**
    - **Draw condition**
4. If the game ends, a **message box appears showing the result**.
5. The game board is then reset for a new game.

---

6. Restart Function

The program includes a **Restart Game button** that:

- Clears the board
- Resets the players
- Enables all buttons again

This allows the game to start again without restarting the program.

---

7. Purpose of the Program

The main purpose of this program is to demonstrate how an **evaluation function (heuristic function)** can be used in board games to determine the value of a game state. It also shows how

Artificial Intelligence concepts can be combined with GUI programming to build an interactive application.

8. Code

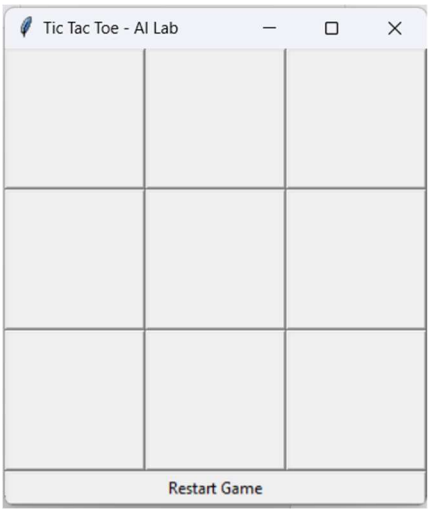
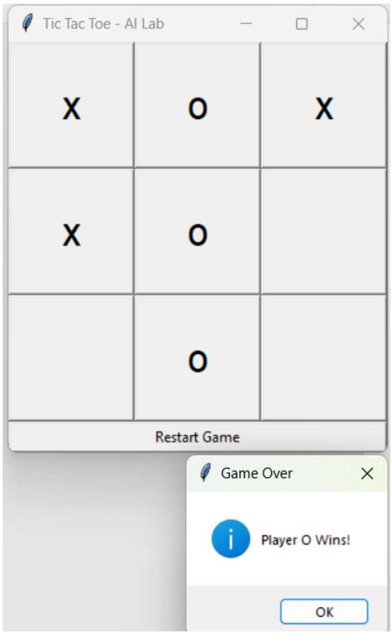
```
import tkinter as tk

from tkinter import messagebox



# Board initialization

board = [['_','_','_'],

         ['_','_','_'],

         ['_','_','_']]


current_player = 'x'




# Evaluation Function

def evaluate(b):



    # Check rows

    for row in range(3):

        if b[row][0] == b[row][1] == b[row][2] and b[row][0] != '_':

            if b[row][0] == 'x':

                return 10

            else:

                return -10
```

```
# Check columns
for col in range(3):
    if b[0][col] == b[1][col] == b[2][col] and b[0][col] != '_':
        if b[0][col] == 'x':
            return 10
        else:
            return -10

# Check diagonals
if b[0][0] == b[1][1] == b[2][2] and b[0][0] != '_':
    if b[0][0] == 'x':
        return 10
    else:
        return -10

if b[0][2] == b[1][1] == b[2][0] and b[0][2] != '_':
    if b[0][2] == 'x':
        return 10
    else:
        return -10

return 0
```

```
# Check if moves left
def is_moves_left(b):
    for i in range(3):
        for j in range(3):
            if b[i][j] == '_':
                return True
    return False



# Button click handler
def click(row, col):
    global current_player

    if board[row][col] == '_':

        board[row][col] = current_player
        buttons[row][col]["text"] = current_player

        score = evaluate(board)

        if score == 10:
            messagebox.showinfo("Game Over", "Player X Wins!")
            reset_game()
            return
```

```
        elif score == -10:
            messagebox.showinfo("Game Over", "Player O Wins!")
            reset_game()
            return

        elif not is_moves_left(board):
            messagebox.showinfo("Game Over", "Draw!")
            reset_game()
            return

        # Switch player
        if current_player == 'x':
            current_player = 'o'
        else:
            current_player = 'x'



# Reset game
def reset_game():
    global board, current_player

    board = [['_','_','_'],
             ['_','_','_'],
```

```
            ['_','_','_']]

    current_player = 'x'

    for i in range(3):
        for j in range(3):
            buttons[i][j]["text"] = ""


# GUI
root = tk.Tk()
root.title("Tic Tac Toe - AI Lab")

buttons = []

for i in range(3):
    row = []
    for j in range(3):
        btn = tk.Button(root,
                        text="",
                        font=("Arial", 25),
                        width=5,
                        height=2,
                        command=lambda r=i, c=j: click(r, c))
```

```
        btn.grid(row=i, column=j)
        row.append(btn)
    buttons.append(row)

reset_btn = tk.Button(root, text="Restart Game", command=reset_game)
reset_btn.grid(row=3, column=0, columnspan=3, sticky="nsew")

root.mainloop()
```

9. Output